

Nome: Henrique Fragoso Fagundes

Turma: 203

Desafio 1 - Tela de Login

1. Qual é o problema apresentado?

O problema apresentado é que a nossa tela de login permite várias tentativas incorretas sem nenhum bloqueio. Isso pode deixar o sistema menos seguro, porque uma pessoa poderia tentar várias senhas até conseguir acertar.

2. Que parte do código está faltando?

Está faltando uma parte do código que conte quantas vezes o usuário errou a senha e que registre o tempo em que ele deve ficar bloqueado após 3 tentativas incorretas. Também precisamos adicionar no banco de dados campos para guardar essas informações.

3. Qual estrutura de programação ou estrutura de dados deve ser utilizada?

Nós utilizaríamos um contador de tentativas, com uma variável do tipo `int`, para contar os erros. Também usaríamos uma variável do tipo `long` para registrar o tempo de bloqueio com `System.currentTimeMillis()`. No banco SQLite, usaríamos as colunas `tentativas` e `bloqueado_ate`.

4. Como a dupla completaria o trecho?

Nós completariamos o trecho fazendo o sistema buscar o usuário no banco de dados antes de validar a senha. Depois, verificaríamos se ele está bloqueado. Se ainda estiver dentro do tempo de bloqueio, o login não seria permitido. Caso não esteja bloqueado, o sistema compararia a senha digitada com a senha cadastrada. Se a senha estivesse correta, as tentativas seriam zeradas. Se estivesse errada, o contador aumentaria. Ao chegar em 3 erros, o sistema salvaria o horário atual mais 3 minutos no campo `bloqueado_ate`.

Exemplo resumido:

```
Usuario usuario = usuarioDAO.buscarPorUsuario(usuarioDigitado);

if (usuario == null) {
    JOptionPane.showMessageDialog(this, "Usuário ou senha incorretos.");
    return;
}

long segundosRestantes = usuarioDAO.segundosRestantesBloqueio(usuario);

if (segundosRestantes > 0) {
    JOptionPane.showMessageDialog(this,
        "Login bloqueado. Tente novamente em " + segundosRestantes + " segundos.");
    return;
}

if (usuario.getSenha().equals(senhaDigitada)) {
    usuarioDAO.zerarTentativas(usuarioDigitado);
    JOptionPane.showMessageDialog(this, "Login realizado com sucesso!");
} else {
    int tentativas = usuarioDAO.registrarTentativaIncorreta(usuarioDigitado);

    if (tentativas >= 3) {
        JOptionPane.showMessageDialog(this,
            "Você errou 3 vezes. O login foi bloqueado por 3 minutos.");
    } else {
        JOptionPane.showMessageDialog(this,
            "Senha incorreta. Tentativa " + tentativas + " de 3.");
    }
}
}
```

5. Por que essa solução é adequada?

Essa solução é adequada porque deixa o sistema mais seguro, impedindo que alguém tente várias senhas seguidas. Além disso, como nós salvamos as tentativas e o tempo de bloqueio no banco SQLite, o bloqueio continua funcionando mesmo se o programa for fechado e aberto novamente.

Desafio 2 - Loja Online

1. Qual é o problema apresentado?

O problema apresentado é que a loja online já exibe os produtos, mas a função de carrinho de compras ainda não foi implementada. Quando o usuário clica em “Adicionar ao Carrinho”, o sistema deveria guardar o produto escolhido, permitir alterar a quantidade, remover itens, calcular o total e finalizar a compra, mas essa parte estava incompleta.

2. Que parte do código está faltando?

A parte que está faltando é a lógica dentro do método adicionarAoCarrinho. Esse método precisava capturar o ID do produto pela URL, procurar o produto na lista, armazenar o item no carrinho, verificar se ele já existia, aumentar a quantidade, mostrar os itens adicionados, calcular o total e criar opções para remover, alterar quantidade e finalizar a compra.

3. Qual estrutura de programação ou estrutura de dados deve ser utilizada?

Nós utilizamos um Map, mais especificamente um HashMap, para representar o carrinho. Nesse HashMap, o ID do produto fica como chave e a quantidade fica como valor. Também usamos estruturas condicionais, como if e else, para verificar a ação escolhida, e laços de repetição for para procurar os produtos na lista.

4. Como a dupla completaria o trecho?

Nós completariamos o trecho somente dentro do método adicionarAoCarrinho, sem alterar o restante do código. Primeiro, criaríamos ou recuperariamos o carrinho usando um HashMap. Depois, pegariamos os parâmetros enviados pela URL, como id, acao e quantidade. Em seguida, procuraríamos o produto dentro da lista produtos.

Se a ação fosse adicionar, o sistema colocaria o produto no carrinho ou aumentaria sua quantidade. Se a ação fosse remover, o produto seria retirado. Se a ação fosse alterar, a quantidade seria atualizada. Se a ação fosse finalizar, o carrinho seria esvaziado e uma mensagem de compra finalizada seria exibida.

Exemplo resumido:

```
java.util.Map<String, Object> atributos = exchange.getHttpContext().getAttributes();
```

```
java.util.Map<Integer, Integer> carrinho =  
    (java.util.Map<Integer, Integer>) atributos.get("carrinho");
```

```
if (carrinho == null) {  
    carrinho = new java.util.HashMap<>();  
    atributos.put("carrinho", carrinho);  
}
```

```
String query = exchange.getRequestURI().getQuery();
```

```
int id = 0;
```

```
if (query != null) {  
    String[] partes = query.split("=");  
  
    if (partes.length == 2 && partes[0].equals("id")) {  
        id = Integer.parseInt(partes[1]);  
    }  
}
```

```
Produto produtoEncontrado = null;
```

```
for (Produto produto : produtos) {  
    if (produto.getId() == id) {  
        produtoEncontrado = produto;  
        break;  
    }  
}
```

```
if (produtoEncontrado != null) {  
    int quantidadeAtual = carrinho.getOrDefault(id, 0);  
    carrinho.put(id, quantidadeAtual + 1);  
}
```

```
double total = 0;
```

```
for (Integer idProduto : carrinho.keySet()) {  
    Produto produto = null;  
  
    for (Produto p : produtos) {  
        if (p.getId() == idProduto) {  
            produto = p;  
            break;  
        }  
    }
```

```
}  
  
if (produto != null) {  
    int quantidade = carrinho.get(idProduto);  
    total += produto.getPreco() * quantidade;  
}  
}
```

5. Por que essa solução é adequada?

Essa solução é adequada porque completa exatamente a parte que estava faltando no código, sem precisar modificar outras partes do programa. O uso do HashMap facilita o controle do carrinho, pois cada produto fica ligado ao seu ID e à sua quantidade. Assim, quando o usuário adiciona o mesmo doce mais de uma vez, o sistema apenas aumenta a quantidade, em vez de criar itens repetidos.

Além disso, a solução permite visualizar o carrinho, calcular o total da compra, alterar quantidades, remover produtos e finalizar a compra, deixando a loja online mais completa e funcional.

Desafio 3 - Clínica Web

1. Qual é o problema apresentado?

O problema apresentado é que a lista de pacientes estava sendo exibida em ordem aleatória, usando uma lógica que embaralhava os cadastros. Isso não representa corretamente uma fila de atendimento, porque em uma clínica o primeiro paciente cadastrado deve ser o primeiro a ser atendido.

2. Que parte do código está faltando?

Está faltando substituir a lógica aleatória por uma lógica de fila. O método `listarPacientesAleatorios` estava criando uma cópia da lista e usando `Collections.shuffle()`, deixando os pacientes fora da ordem correta. Era necessário alterar essa parte para seguir o modelo FIFO.

3. Qual estrutura de programação ou estrutura de dados deve ser utilizada?

Nós utilizamos uma fila, representada por `Queue`, com `LinkedList`. Essa estrutura segue o modelo FIFO, que significa First In, First Out, ou seja, o primeiro que entra é o primeiro que sai. Também usamos laços de repetição para passar os pacientes da lista para a fila e depois exibir na ordem correta.

4. Como a dupla completaria o trecho?

Nós completaríamos o trecho criando uma fila de pacientes. Depois, adicionaríamos todos os pacientes cadastrados nessa fila usando `add()`. Em seguida, passaríamos os pacientes da fila para uma nova lista usando `poll()`, mantendo a ordem de chegada.

Exemplo resumido:

```
java.util.Queue<Paciente> fila = new java.util.LinkedList<>();

for (Paciente paciente : pacientes) {
    fila.add(paciente);
}

List<Paciente> listaFIFO = new ArrayList<>();

while (!fila.isEmpty()) {
    listaFIFO.add(fila.poll());
}

return listaFIFO;
```

5. Por que essa solução é adequada?

Essa solução é adequada porque representa corretamente uma fila de atendimento. Com o modelo FIFO, o primeiro paciente cadastrado aparece primeiro na lista, respeitando a ordem de chegada. Além disso, a solução substitui a parte aleatória do código por uma estrutura própria para filas, deixando o sistema mais correto e organizado.

Desafio 4 - Loja de Carros Web

1. Qual é o problema apresentado?

O problema apresentado é que o histórico de páginas visitadas estava usando ArrayList, que funciona apenas como uma lista comum. Porém, para simular o botão de voltar página, o comportamento correto deveria ser de pilha, onde a última página acessada é a primeira a sair.

2. Que parte do código está faltando?

Está faltando trocar a estrutura de lista por uma estrutura de pilha. Também era necessário alterar os métodos que registram e removem páginas, substituindo add() por push() e a remoção manual do último item por pop().

3. Qual estrutura de programação ou estrutura de dados deve ser utilizada?

Nós utilizamos uma pilha com Deque e ArrayDeque. Essa estrutura segue o modelo LIFO, que significa Last In, First Out, ou seja, o último que entra é o primeiro que sai. Para isso, usamos os métodos push(), pop(), peek() e isEmpty().

4. Como a dupla completaria o trecho?

Nós completamos o trecho trocando o histórico de List para Deque. Depois, no método registrarPagina, usaremos push() para colocar a página no topo da pilha. No método voltarPagina, verificaremos se o histórico está vazio e, se não estiver, usamos pop() para remover e retornar a última página acessada.

Exemplo resumido:

```
private static Deque<String> historico = new ArrayDeque<>();

public static void registrarPagina(String pagina) {
    historico.push(pagina);
}

public static List<String> listarHistorico() {
    return new ArrayList<>(historico);
}

public static String voltarPagina() {
    if (historico.isEmpty()) {
        return null;
    }

    return historico.pop();
}
```

5. Por que essa solução é adequada?

Essa solução é adequada porque o botão de voltar página funciona como uma pilha. A última página visitada deve ser a primeira a ser removida quando o usuário clica em voltar. Assim, o uso de Deque com ArrayDeque deixa o sistema mais correto, pois aplica o modelo LIFO pedido na atividade.

Desafio 5 - Cadastro Web

1. Qual é o problema apresentado?

O problema apresentado é que o sistema permitia cadastrar pessoas com dados repetidos, como CPF e e-mail. Isso pode causar confusão no sistema, porque duas pessoas poderiam aparecer com a mesma identificação, mesmo que o CPF e o e-mail devam ser únicos.

2. Que parte do código está faltando?

Está faltando uma validação antes de salvar ou editar o cadastro. O sistema precisava verificar se já existia uma pessoa cadastrada com o mesmo CPF ou com o mesmo e-mail. Caso existisse, o cadastro não deveria ser salvo e uma mensagem de erro deveria ser exibida.

3. Qual estrutura de programação ou estrutura de dados deve ser utilizada?

Nós utilizamos a própria lista de pessoas cadastradas, percorrendo os cadastros com um laço for. Também usamos estruturas condicionais, como if, para comparar o CPF e o e-mail digitados com os dados que já existem no sistema.

4. Como a dupla completaria o trecho?

Nós completariamos o trecho criando uma verificação antes de adicionar uma nova pessoa. O sistema passaria pela lista de pessoas e compararia o CPF e o e-mail. Se encontrasse um cadastro com o mesmo CPF ou e-mail, retornaria uma mensagem de erro e não salvaria o novo cadastro.

Exemplo resumido:

```
public static boolean cpfOuEmailJaExiste(String cpf, String email) {  
  
    for (Pessoa pessoa : pessoas) {  
  
        if (pessoa.getCpf().equals(cpf)) {  
            return true;  
        }  
  
        if (pessoa.getEmail().equals(email)) {  
            return true;  
        }  
    }  
  
    return false;  
}  
  
if (cpfOuEmailJaExiste(pessoa.getCpf(), pessoa.getEmail())) {  
    return "Erro: CPF ou e-mail já cadastrado.";  
}  
  
pessoas.add(pessoa);
```

5. Por que essa solução é adequada?

Essa solução é adequada porque impede que o sistema tenha cadastros repetidos com o mesmo CPF ou e-mail. Isso deixa os dados mais organizados e confiáveis, já que cada pessoa deve ter uma identificação única. Além disso, a validação evita erros tanto no cadastro quanto na edição de informações.